

- 人工智能实验报告 第6周
 - 任务三：
 - 一.实验题目：遗传算法解决旅行商问题
 - 二.实验内容
 - 1.算法原理
 - 2.关键代码展示(可选)
 - 三.实验结果及分析
 - 1.实验结果展示示例（可截图可表可文字，尽量可视化）
 - 2.评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）

人工智能实验报告 第6周

姓名:陈安康 学号:22320021

任务三：

一.实验题目：遗传算法解决旅行商问题

二.实验内容

1.算法原理

遗传算法是一种模仿生物进化过程的优化算法。它基于达尔文的进化论和孟德尔的遗传学原理，通过模拟自然选择、交叉和变异过程来搜索最优解。而旅行商问题是一个经典的NP-hard的问题，其描述为：给定一组城市和每对城市之间的距离，求解访问每个城市一次并回到起始城市的最短路径。通过遗传算法去解决TSP问题，算法主要的操作步骤如下：

1.读取TSP文件，初始化城市位置信息和通过随机生成序列来初始化种群，在这里是随机生成的不同路径的集合。

2.在给定迭代次数下，计算种群中每个个体的适应度（这里就是不同路径的总距离，这里规定的是总距离越小适应度越大），依据适应度的大小以相应的概率选择两个个体作为父代进行交叉操作，得到的子代依据概率进行变异操作，判定得到的子代是否存在于种群中，若没有，加入种群。

3.依据适应度选择适应度更大的个体组成新的种群，判断是否达到迭代次数，若达到，结束算法，返回目前种群中的最优个体，否则，重复操作2。

对于算法的优化：

遗传算法运行性能主要和变异率的设置（我的算法里面没有设置交叉率，默认一定会交叉生成子代）、迭代次数、种群大小、交叉和变异等操作的算法选择有关。在实际运行过程中我进行了各种尝试，大部分都以失败告终，不过感觉应该呈现出来，失败的优化想法在代码中都是注释掉的代码行。

1.对于交叉操作，课件中讲述的是映射交叉，在实际使用过程中，我发现对于合成的子代，在对其进行赋值时，重复的部分直接由父代非重复的部分填充即可，简单粗暴效果好，即节省的操作时间（微不足道），在性能上也稍微优于映射交叉，至于原因应该是这种操作方法产生的子代相比于映射操作更加随机，更能在后面打破局部最优的限制，找出更好的解。

2.在观察适应度（在这里是总距离）收敛图后我发现，在迭代的后期收敛速度十分的慢，整个函数的迭代曲线与负指数函数一致，以至于后期无论加大多少次迭代次数，因为其收敛愈加缓慢的特点都得不到很好的效果，而为了跳出局部最优解，我的想法是在变异率上着手改进，从起初的静态变异率到设置动态变异率。最初我的想法是设置变异率随着迭代次数线性减小，可是在实际应用中与静态变异率的运行效果相差无几，由此我改变思路，将变异率的增减和适应度挂钩，通过适应度动态调整变异率。做法如下：设置一个计数器，如果前两次迭代结果变化幅度过小，则计数器加一，若计数器达到指定的阈值，则切换模式，以高变异率进行迭代，直到变化幅度达到预期值，计数器清零，退出高变异率模式，转为低变异率进行迭代。其中高变异率模式下变异率随迭代次数线性增加，正常模式下变异率随迭代次数线性减小。这么做有一些好处，其中一个就是为了防止在迭代初期算法误入高变异率模式后因为变异率过高而导致初期收敛效果受到影响（因为初期高变异率模式下的变异率也不会很高），而正常模式下变异率随迭代次数线性减小可以保证结果收敛。经过验证，这种策略可以保证在迭代后期数据收敛速度不至于停步不前，效果上要比静态变异率的情况下好一些。

3.对于迭代算法的实现流程上，起初是有两种实现方法，一种是将变异操作仅施加与子代身上，另一种是交叉操作施加与全体种群，对于施加个体的选择，则选择适应度比较低的个体，期望通过变异得到更好的个体，从而使结果更优。可是实际过程中发现效果并不是很好，分析原因认为是变异引起的改变太有限了，其效果只能影响较小范围的城市范围，不足以让低适应度个体完成“蜕变”，从而在选择过程中还是要被淘汰。从而浪费了变异的操作，导致未能达到预期效果。

变异率动态变化函数如下：

```
# 变化率从0.1 - 0.3
def rate_reversemutation(self, count):
    mutation_rate = 0.3 - 0.2 * (float(count) / self.num_iterations)
    # c = min(self.population, key = self.sort_by_reference)
    # a = self.population_dist[self.population.index(c)]
    # b = 2 * a
    a = 1
    b = 2
    if len(self.ans_distance) >= 2:
        b = self.ans_distance[-2]
        a = self.ans_distance[-1]
    if abs((b - a) / b) < 0.00001:
        self.mutation_times_reverse = self.mutation_times_reverse + 1
    if self.mutation_times_reverse > self.num_iterations * 0.3 +
self.num_iterations - count:
        mutation_rate = 0.3 + 0.2 * (float(count) / self.num_iterations)
        #输出一些信息，方便debug的
        print(count)
        print(abs((b - a) / b))
        print(a)
        if abs((b - a) / b) >= 0.001:
            self.mutation_times_reverse = 0
    return mutation_rate
```

求解函数代码如下：

```
def iterate(self, num_iterations):
    self.num_iterations = num_iterations
    for c in range(num_iterations):
        new_population = []
        for i in range(self.operate_num):
            #a,b = random.sample(self.population, 2)
            a, b = self.select_parent()
            child1, child2 = self.crossover(a, b)
            #child1 = self.crossover(a, b)
            if random.random() < self.rate_swapmutation(c):
                # if random.random() < 0.3:
                child1 = self.swapmutate(child1)
            if random.random() < self.rate_reversemutation(c):
                # if random.random() < 0.3:
                child1 = self.reversemutae(child1)
            if random.random() < self.rate_swapmutation(c):
                #if random.random() < 0.3:
                child2 = self.swapmutate(child2)
            if random.random() < self.rate_reversemutation(c):
                #if random.random() < 0.3:
                child2 = self.reversemutae(child2)
            if child1 not in self.population:
                self.population.append(child1)
                self.population_dist.append(self.distance(child1))
```

```

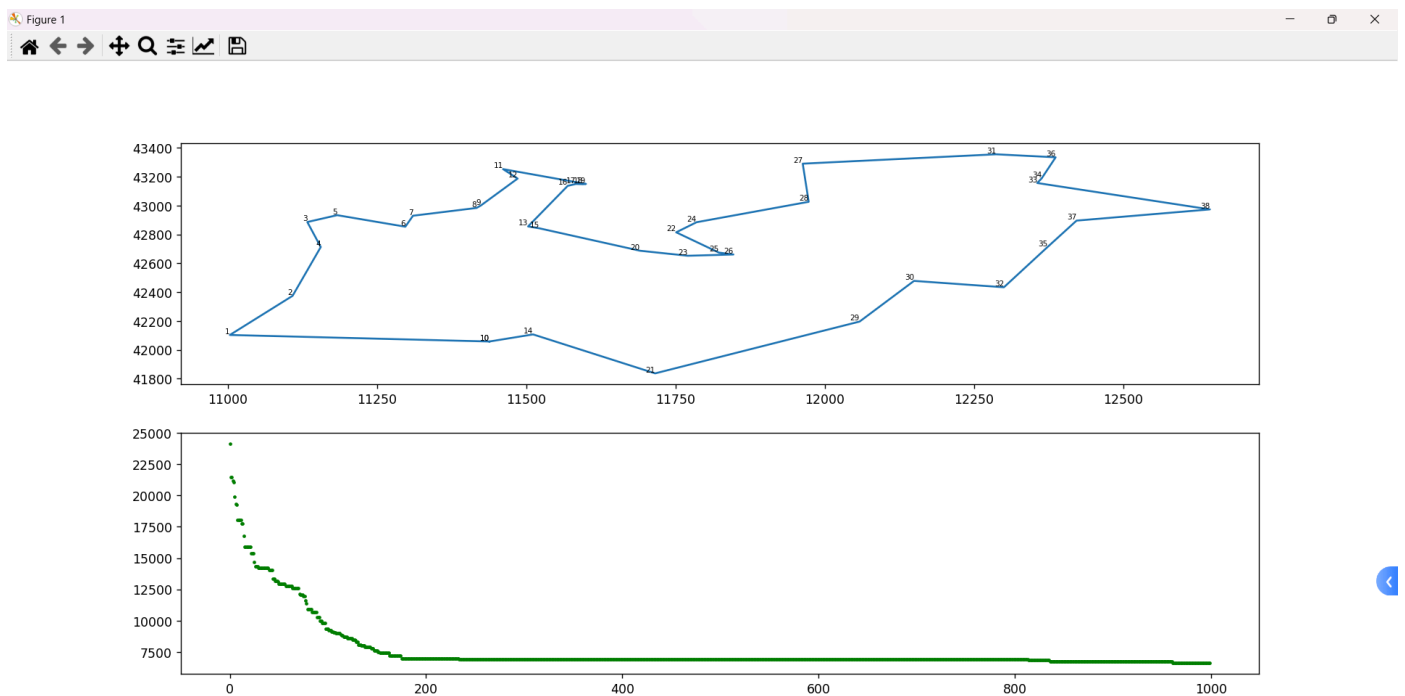
        if child2 not in self.population:
            self.population.append(child2)
            self.population_dist.append(self.distance(child2))
        self.population = self.select()
        self.update_dist()
        best_individual = min(self.population, key = self.sort_by_reference)
        best_val = self.distance(best_individual)
        self.ans_distance.append(best_val)
        # print(i)
        #best_individual = min(self.population, key = lambda x : self.distance(x))
        #best_val = self.distance(best_individual)
        ans_best_individual = [x + 1 for x in best_individual]
        return ans_best_individual, best_val

```

三.实验结果及分析

1.实验结果展示示例（可截图可表可文字，尽量可视化）

在指定网站下载tsp文件后进行测试，对于城市数目不过百的数据，迭代次数为1000（其实有时候500-800就能跑出来，不过看概率，1000保守点），种群规模设置为20时基本上就能求出最优解。（下图城市数目38，最优路径是6656）



```

0.0
6659.431532931466
[10, 1, 2, 4, 3, 5, 6, 7, 8, 9, 12, 11, 19, 18, 17, 16, 13, 15, 20, 23, 26, 25, 22, 24, 28, 27, 31, 36, 34, 33, 38, 37, 35, 32, 30, 29, 21, 14, 10]
6659.431532931466

```

对于数据为200 - 300 的数据，迭代次数为6000-8000，种群数目为20时，求出的计算结果与最优解接近，再往上增加迭代次数，结果会逐步接近最优解，但优化效果并不明显，收敛速度会变得十分慢。（下图城市数目237，最优路径是1019）

Figure 1 displays two plots illustrating the evolution of a network over time.

The top plot shows a network graph with nodes (represented by numbers) and edges (represented by lines). The nodes are distributed across the horizontal axis (0 to 120) and the vertical axis (0 to 60). The network structure is complex, with many nodes and edges, suggesting a highly interconnected system.

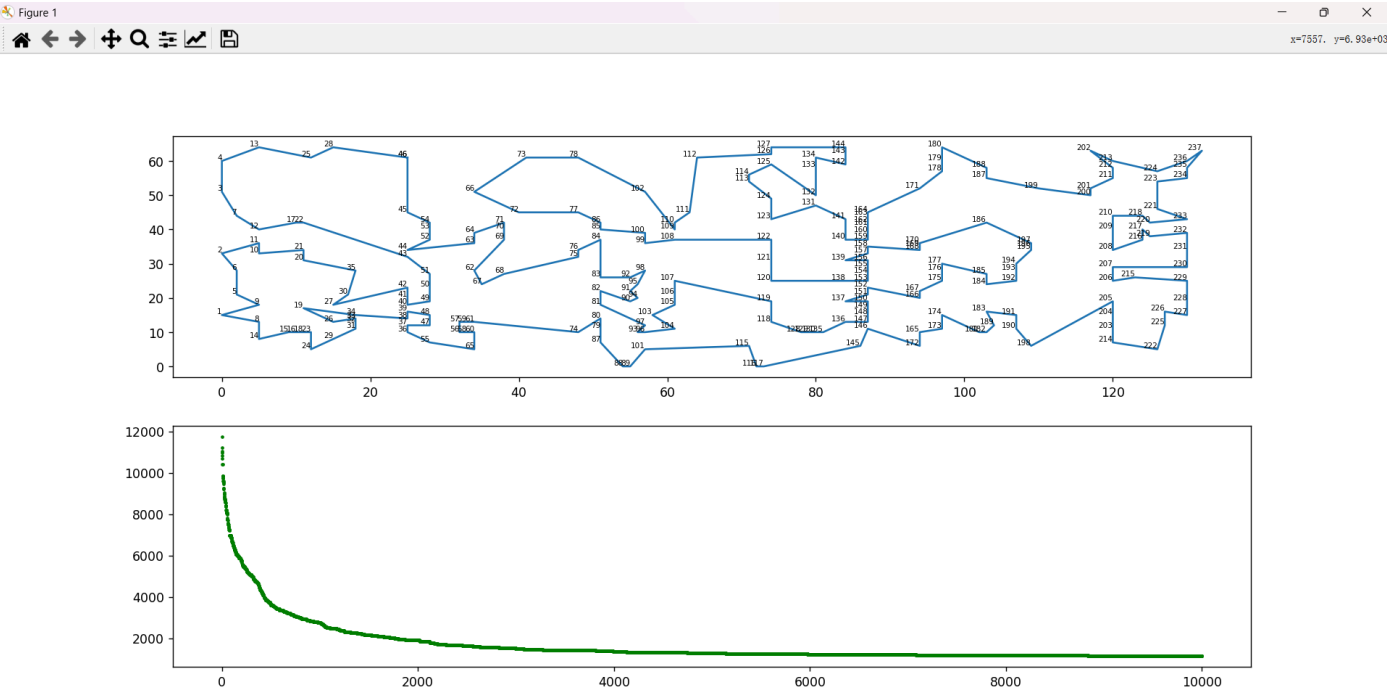
The bottom plot shows the number of nodes (Y-axis, 0 to 10000) versus time (X-axis, 0 to 6000). The curve starts at approximately 10000 nodes at time 0 and decreases rapidly, following an exponential-like decay, reaching a stable state of approximately 1000 nodes by time 6000.

[236, 235, 224, 211, 201, 200, 217, 208, 216, 219, 232, 231, 230, 215, 206, 205, 207, 197, 186, 171, 142, 143, 144, 134, 127, 126, 125, 114, 113, 112, 102, 111, 124, 123, 122, 110, 109, 99, 108, 100, 84, 85, 86, 77, 72, 70, 71, 66, 78, 73, 46, 25, 37, 13, 28, 4, 3, 7, 21, 20, 30, 35, 43, 44, 52, 63, 69, 64, 53, 54, 45, 22, 17, 12, 11, 10, 2, 6, 5, 1, 14, 16, 18, 24, 29, 23, 8, 9, 19, 27, 26, 34, 33, 32, 31, 36, 37, 55, 56, 47, 48, 38, 39, 40, 49, 41, 42, 50, 51, 62, 67, 61, 59, 57, 58, 60, 65, 74, 79, 87, 88, 89, 101, 96, 93, 80, 90, 81, 82, 68, 75, 76, 83, 92, 98, 95, 91, 94, 97, 104, 103, 105, 106, 107, 121, 120, 119, 138, 154, 153, 152, 151, 137, 150, 149, 148, 147, 146, 136, 135, 130, 129, 128, 118, 115, 116, 117, 145, 172, 165, 173, 174, 166, 167, 175, 155, 139, 156, 157, 158, 169, 168, 177, 176, 184, 191, 183, 189, 181, 182, 19, 0, 198, 214, 203, 222, 225, 227, 228, 229, 226, 204, 192, 185, 193, 194, 196, 195, 170, 159, 160, 140, 141, 131, 132, 133, 164, 163, 162, 161, 178, 180, 188, 187, 1, 99, 209, 212, 232, 234, 221, 233, 220, 218, 202, 213, 212, 237, 236]

1329.5104438560902

以上结果均运行过多次，选出的最一般情况作为展示，可以看到正常模式与高变异率模式下变异率均随迭代次数动态变化的运行结果更加好一些。

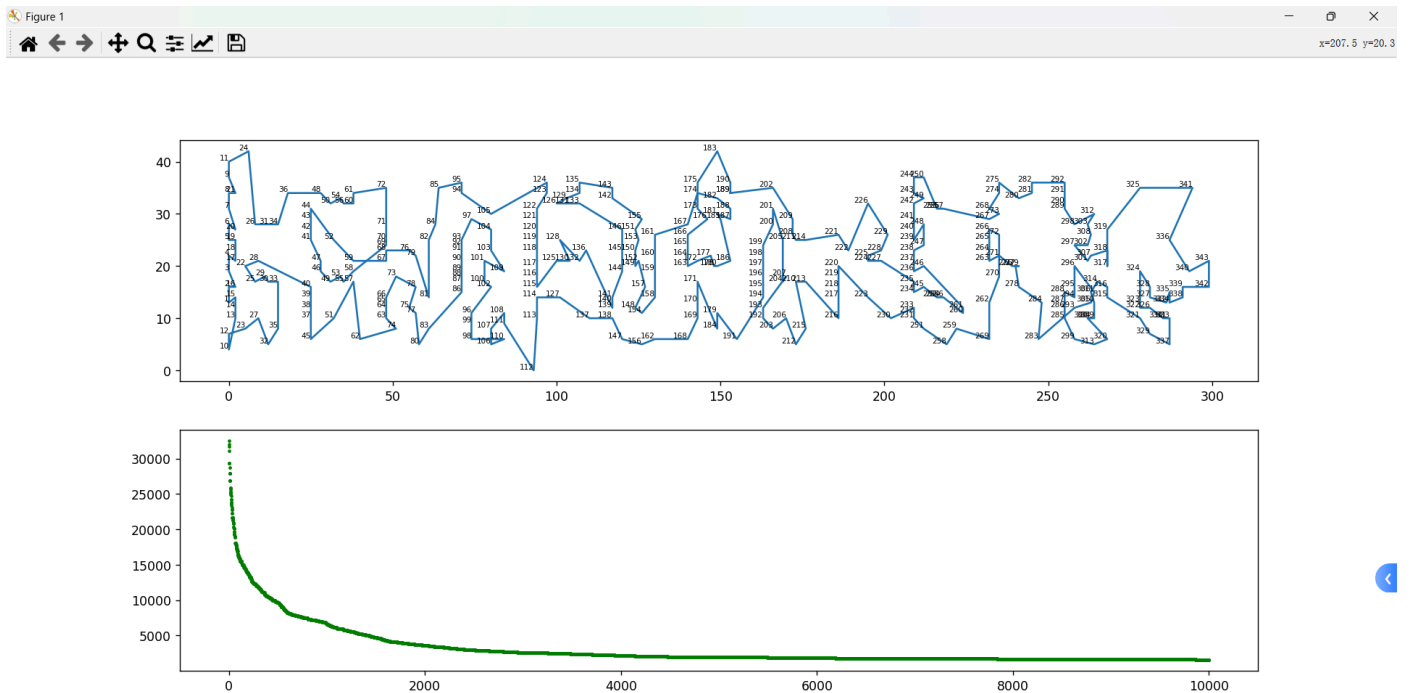
以下是迭代次数为10000，种群大小为20 时，237个节点运行结果，可以看到后面也更加收敛到最优结果：



```
[46, 45, 54, 53, 52, 44, 63, 64, 71, 70, 69, 62, 67, 68, 75, 76, 84, 83, 92, 98, 95, 91, 94, 90, 82, 81, 97, 93, 96, 104, 103, 105, 106, 107, 119, 118, 128, 129, 130, 135, 136, 147, 148, 149, 150, 137, 151, 152, 166, 167, 175, 176, 177, 185, 184, 192, 193, 194, 195, 196, 197, 186, 170, 169, 168, 158, 157, 139, 156, 155, 154, 153, 138, 120, 121, 122, 108, 99, 100, 85, 86, 77, 72, 66, 73, 78, 102, 109, 110, 111, 112, 126, 127, 144, 143, 142, 134, 133, 132, 125, 114, 113, 124, 123, 131, 141, 140, 159, 160, 161, 162, 163, 164, 171, 178, 179, 180, 188, 187, 199, 200, 201, 211, 212, 202, 213, 224, 236, 237, 235, 234, 223, 221, 233, 220, 218, 210, 209, 208, 216, 217, 219, 232, 231, 230, 207, 206, 215, 229, 228, 227, 226, 225, 222, 214, 203, 204, 205, 198, 190, 191, 183, 189, 182, 181, 174, 173, 165, 172, 146, 145, 117, 116, 115, 101, 89, 88, 87, 79, 80, 74, 61, 59, 57, 56, 58, 60, 65, 55, 36, 37, 47, 48, 39, 38, 34, 19, 26, 33, 32, 31, 29, 24, 23, 18, 16, 15, 14, 8, 1, 9, 5, 6, 2, 11, 10, 21, 20, 35, 30, 27, 42, 41, 40, 49, 50, 51, 43, 22, 17, 12, 7, 3, 4, 13, 25, 28, 46]
```

1161.7398625295987

下图是城市数目为343，迭代次数设置为10000，种群数目为20时的结果图。（官方最优结果是1368）



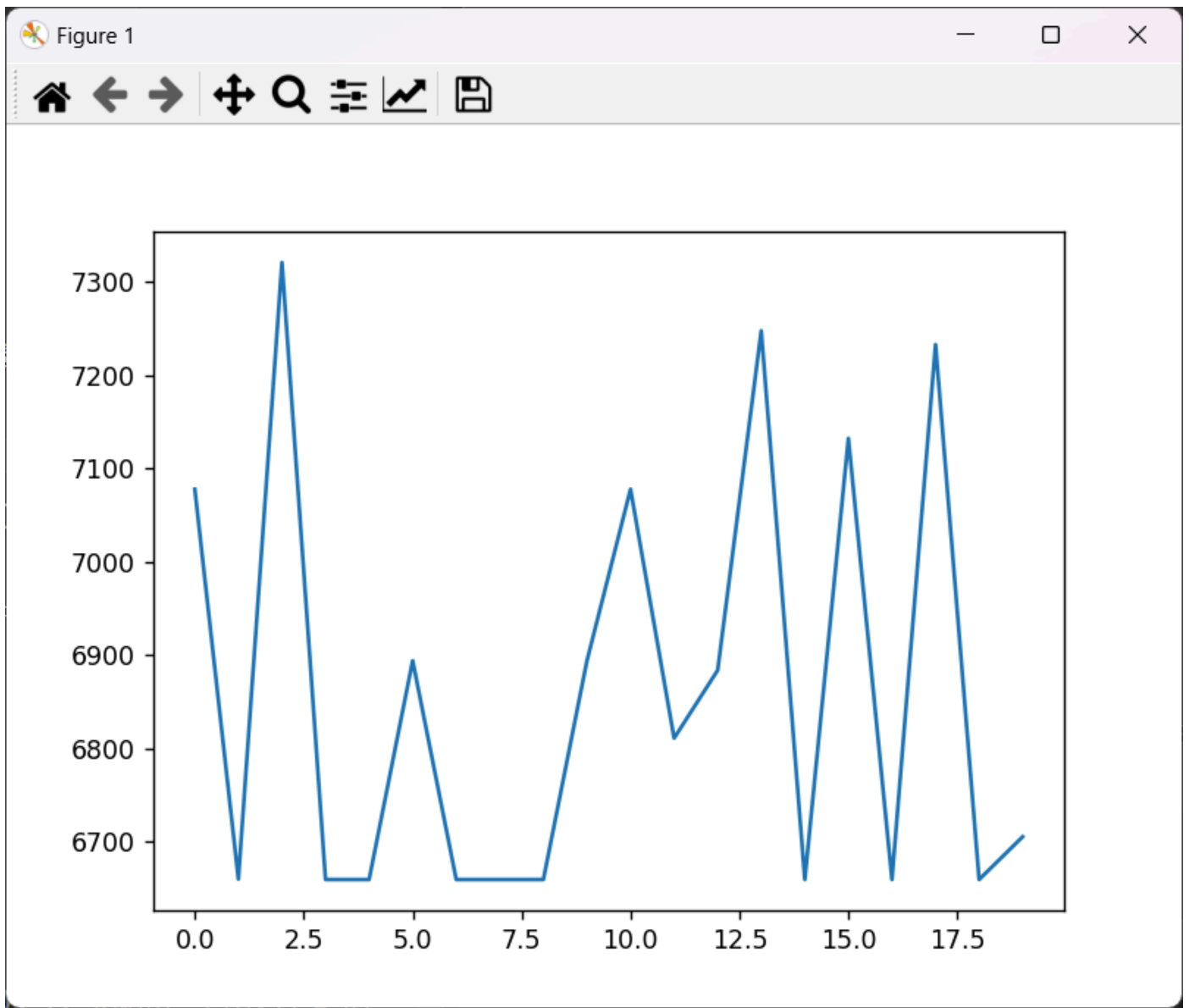
```
[189, 190, 183, 175, 173, 176, 166, 165, 164, 163, 172, 177, 178, 180, 186, 185, 181, 187, 188, 182, 174, 167, 161, 160, 159, 158, 154, 148, 157, 152, 149, 150, 153, 151, 155, 142, 143, 135, 134, 129, 126, 131, 133, 146, 145, 144, 141, 140, 139, 136, 132, 128, 130, 125, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 105, 94, 95, 85, 84, 82, 81, 79, 76, 68, 67, 59, 52, 44, 43, 42, 41, 47, 46, 49, 53, 55, 58, 69, 70, 71, 72, 61, 60, 56, 54, 50, 48, 36, 34, 31, 26, 24, 11, 9, 21, 8, 7, 20, 6, 5, 19, 18, 17, 4, 3, 16, 2, 1, 15, 14, 13, 10, 12, 23, 27, 32, 35, 33, 30, 29, 25, 22, 28, 40, 39, 38, 37, 45, 51, 57, 62, 74, 63, 64, 65, 66, 73, 78, 75, 77, 80, 83, 86, 88, 87, 89, 90, 91, 92, 93, 97, 104, 103, 109, 101, 100, 102, 96, 99, 98, 110, 106, 107, 108, 111, 112, 113, 114, 127, 137, 138, 147, 156, 162, 168, 169, 170, 171, 184, 179, 19, 1, 194, 195, 196, 197, 198, 199, 200, 201, 205, 204, 207, 193, 192, 203, 206, 212, 215, 210, 213, 216, 217, 218, 219, 220, 223, 230, 233, 232, 231, 251, 258, 259, 269, 2, 62, 270, 272, 266, 265, 264, 263, 271, 276, 279, 277, 278, 284, 283, 293, 305, 311, 296, 295, 294, 288, 287, 286, 285, 299, 313, 320, 300, 304, 309, 310, 306, 314, 316, 315, 321, 329, 337, 333, 331, 330, 326, 322, 323, 324, 328, 327, 332, 335, 334, 338, 339, 342, 343, 340, 336, 341, 325, 319, 317, 318, 307, 301, 297, 302, 308, 303, 312, 298, 289, 290, 291, 292, 282, 281, 280, 275, 274, 268, 273, 267, 257, 255, 253, 250, 244, 243, 249, 242, 241, 240, 239, 248, 247, 238, 237, 236, 246, 261, 260, 256, 254, 252, 245, 234, 235, 227, 224, 225, 228, 229, 226, 222, 221, 214, 211, 208, 209, 202, 189]
1595.9882168316988
```

2.评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）

主要评测指标是收敛性、稳定性、精度、适应度评价次数。

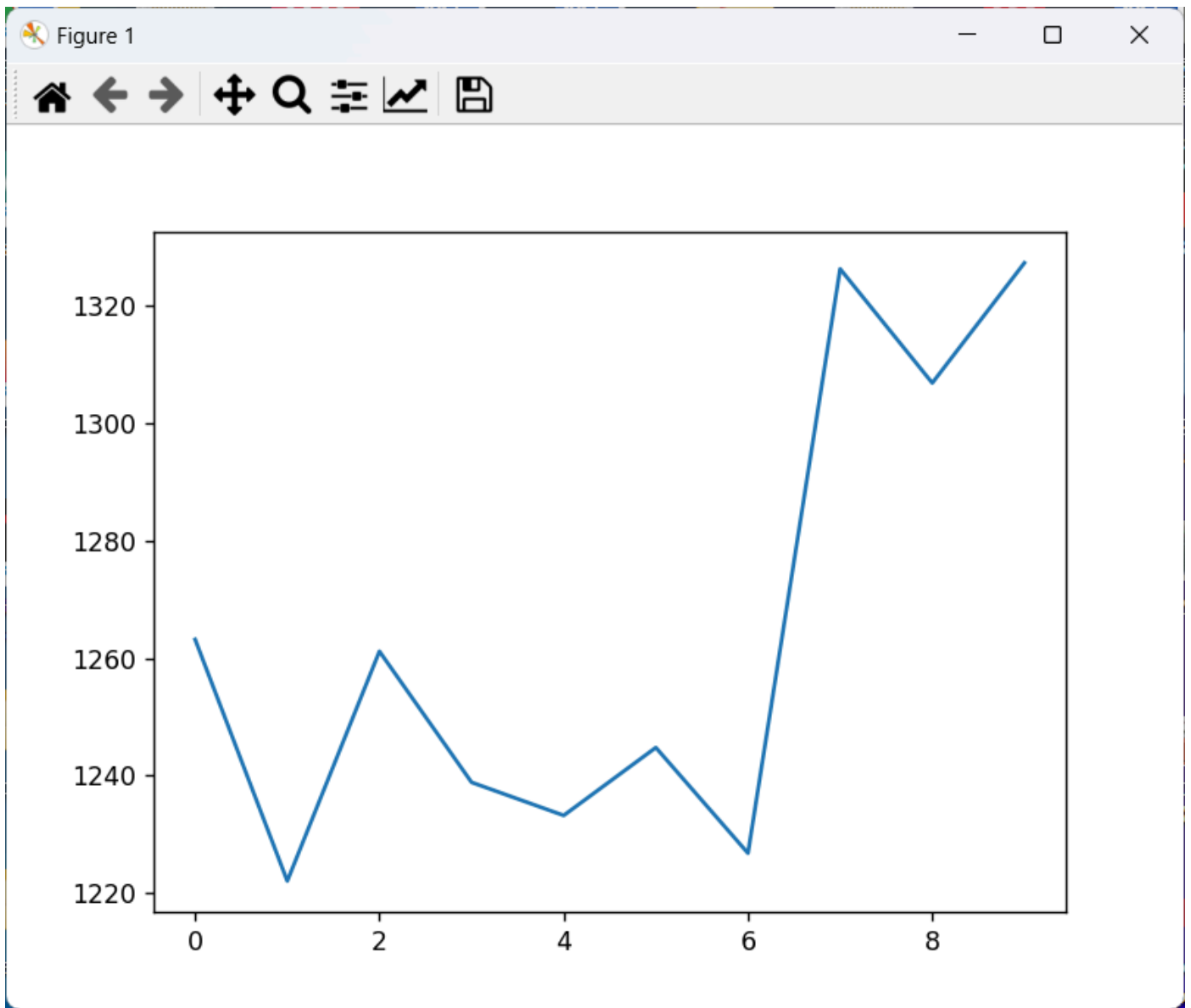
收敛性方面从以上结果展示中可以很明显的看到随着迭代次数的增加，总距离曲线不断地下降，结果不断地收敛到最优结果，收敛性好。

稳定性上，在二十次对城市数目为38的城市进行模拟的结果如下：



可以看到结果大体稳定在6600-7300之间。

以下是上述237个城市在6000迭代次数，20种群大小下的运行10次的结果图。可以看到，结果均稳定在1220-1320的范围内。变化幅度不大。



精度上，在迭代次数和种群等参数设定合适的情况下，进行充分收敛后，结果误差都不会大于最优解的14%，如上面的展示结果。

适应度评价次数即函数`distance`的运行次数，由于函数设置了一个数组对每个个体的`distance`计算结果进行了保存，所以调用适应度评估函数仅在初始化种群，更新种群，加入子代个体时，假设迭代次数记为 $num_{iteration}$ ，规定的种群大小为 pop_{size} ，添加子代的操作要进行 $4pop_{size}$ 次，所以执行次数为 $5pop_{size} * num_{iteration}$ 。